

Projet L'Île Interdite

Par: Maria FERHANI & Malak Yasser Samir ELMASRY

1. Présentation du projet

Ce projet est une reproduction graphique du célèbre jeu coopératif L'Île Interdite, codée en Java avec Swing pour la partie interface graphique.

Le but du jeu est :

- De récupérer 4 artefacts élémentaires (Feu, Eau, Terre, Air),
- Puis de réunir tous les joueurs sur la case de l'héliport pour s'échapper avant que l'île ne soit totalement submergée.

2. Fonctionnalités principales

- La grille est initiée avec les joueurs, et une case attribuée aléatoirement pour chaque artefact, et une pour l'héliport.

On a 3 actions maximum par tour pour chaque joueur, à choisir entre 6 options:

- déplacement rapide par clavier (Q, Z, S, D).
- Assèchement de la même façon avec F pour la case où on se trouve.
- Récupération d'artefacts si 4 clés identiques sont possédées.
- Don de clés entre joueurs présents sur la même case. (avec un choix de la clé qu'on veut donner et le joueur auquel on veut la donner)
- Utilisation de **clés spéciales** : *Sac de Sable* (assèchement libre), *Hélicoptère* (téléportation).
- Fin de tour qui entraîne automatiquement l' **inondation** de 3 cases + **distribution aléatoire d'une carte** (clés d'éléments, ou clés spéciales, ou "montée des eaux" qui submerge la case sur laquelle il se trouve).
- Conditions de **victoire** (4 artefacts + tous à l'héliport) et de **défaite** (case de l'héliport ou un des 4 artefacts submergé ou joueur piégé par cases inondées).

3. Organisation des fichiers

Jeu.java → C'est le fichier principal qui lance la fenêtre du jeu.

FenetreJeu.java → C'est la classe qui crée la fenêtre principale, gère les boutons, le déroulement des tours, et les interactions clavier.

Grille.java → Représente toute l'île sous forme de grille de cases (zones). Contient aussi la liste des joueurs.

Zone.java → Modèle pour chaque case : état (normale, inondée, submergée), artefact éventuel, et si c'est l'héliport.

GrillePanel.java → C'est l'affichage graphique de la grille (zones, joueurs, artefacts, hélicoptère, sac de sable...).

Joueur.java → Modélise un joueur : son nom, sa position, ses clés possédées, ses artefacts récupérés, ses déplacements et ses actions.

Cle.java → Une énumération qui liste tous les types de clés disponibles dans le jeu : AIR, EAU, FEU, TERRE, SAC_DE_SABLE, HELICOPTERE.

Artefact.java → Représente un artefact (associé à un élément : AIR, EAU, FEU, TERRE) que les joueurs doivent récupérer.

PaquetCartes.java

→ Gère le tirage aléatoire des cartes à chaque fin de tour : clés normales, clés spéciales, montée des eaux.

Dossier images/ → Contient toutes les images utilisées par le jeu

JTestUnit.java → contient des tests automatisés JUnit pour valider :

- Le déplacement d'un joueur.
- L'assèchement d'une zone.
- La récupération d'un artefact.
- La détection de la victoire ou défaite.

4. Étapes de développement:

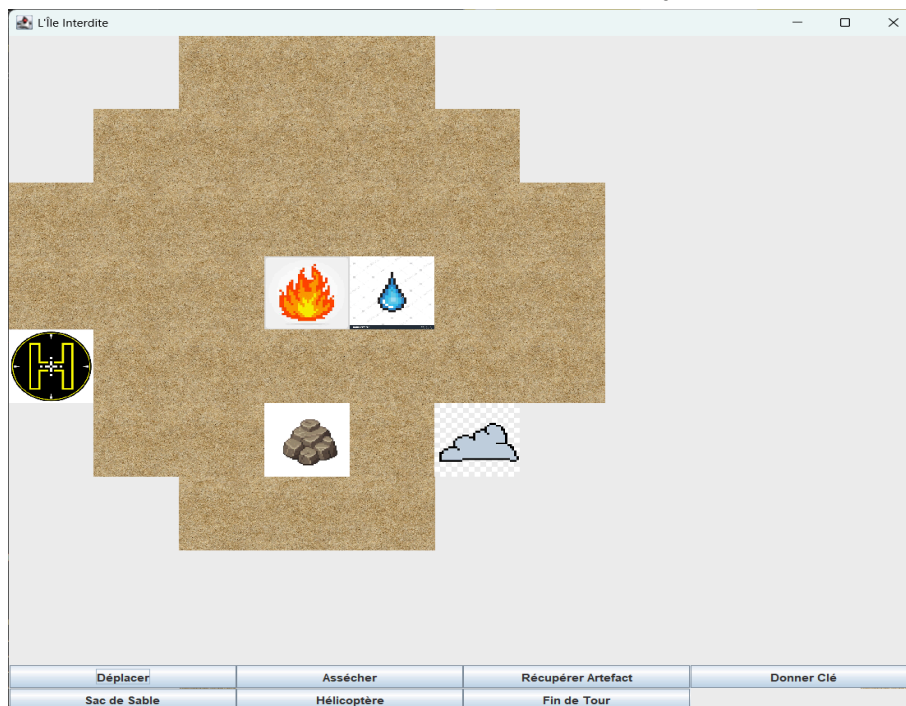


On a commencé par coder les bases du jeu qui s'affichait

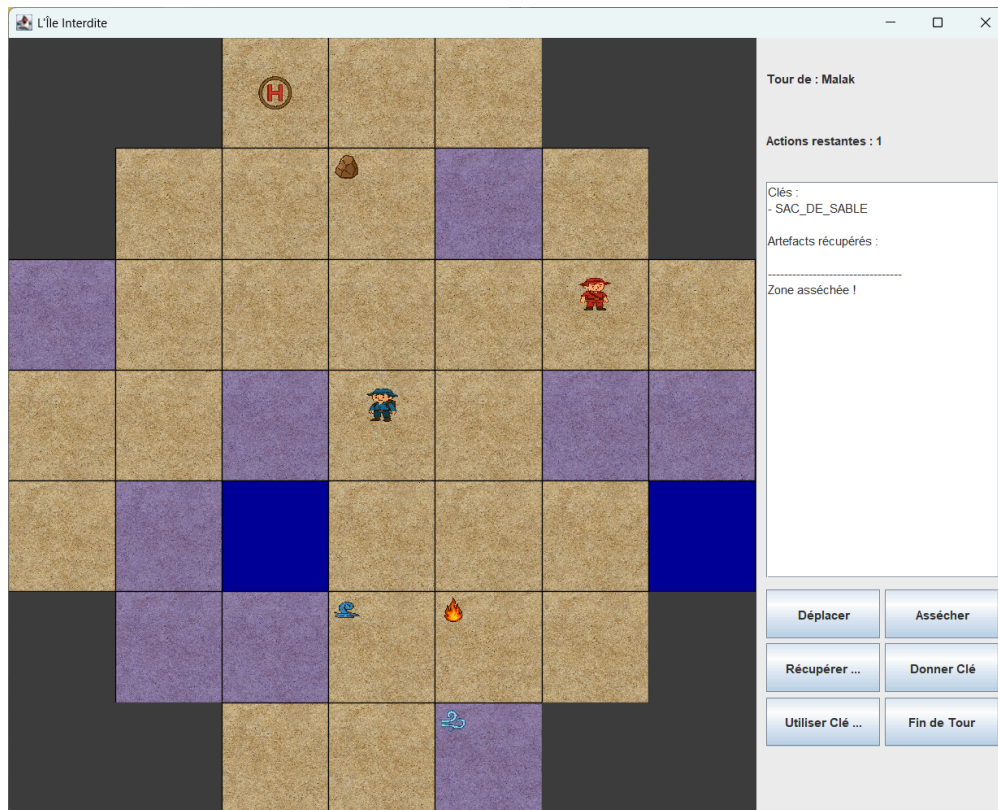
seulement dans terminal, pour nous assurer du fonctionnement logique du jeu, et d'ajouter petit à petit les développements nécessaires sans nous soucier de l'affichage graphique au début.

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... ^ x
Position actuelle : 2,2
Actions restantes : 3
Action ? (1 = Déplacer, 2 = Assécher, 3 = Rien, 4 = Récupérer artefact, 5 = S'envoler, 6 = Donner une clé, 7= Utiliser le sac de sable, 8= Utiliser Hélicoptère : 1
Direction (haut/bas/gauche/droite) : bas
      N N N
    N N N H(N) N
  N N N F(N) N N N
E(N) N N N N A(N) N
N N N N N N N
    N N N N N
      N N T(N)
Position actuelle : 3,2
Actions restantes : 2
Action ? (1 = Déplacer, 2 = Assécher, 3 = Rien, 4 = Récupérer artefact, 5 = S'envoler, 6 = Donner une clé, 7= Utiliser le sac de sable, 8= Utiliser Hélicoptère : 
Ln 4, Col 1 Spaces: 4 UTF-8 CRLF {} Java
```

Et quand on avait tous les éléments qu'on voulait ajouter, on a passé à l'affichage par jpanel:



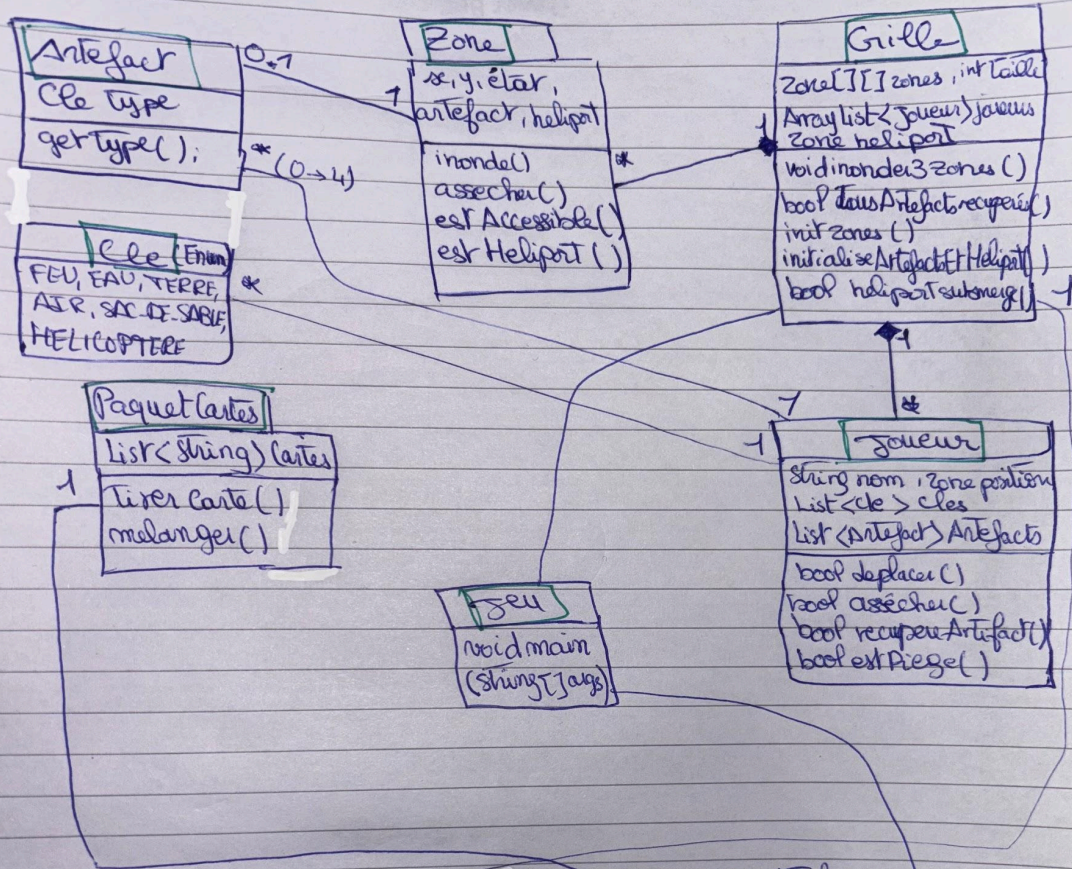
Et on l'a raffiné pour le rendre le plus fonctionnel et clair possible:



5. Diagrammes de classes:

Un résumé de toutes les classes de notre projet, avec leurs attributs et méthodes principaux, et les associations et cardinalités entre eux, selon l'architecture modèle-vue-contrôleur (MVC):

Modèle



Vue

Vue / Contrôleur

